Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

Дисциплина: Интернет–технологии и веб–программирование

ОТЧЁТ
по лабораторной работе №4
« СОЗДАНИЕ АДАПТИВНОГО МАКЕТА С ИСПОЛЬЗОВАНИЕМ *CSS*»
Вариант №8

Выполнили А. Г. Аникейчик
Я.Д. Бельмач

Проверил Н. В. Хаджинова

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 4
2 Краткие Теоретические сведения 5
2.1 Базовый синтаксис *CSS* 5
2.2 Понятие блочной модели 5
2.3 Виды *CSS* макетов 6
2.4 Основы медиазапросов и их применение 7
2.5 Адаптивная вёрстка и принцип *Mobile First* 7
3 Ход работы 9
3.1 Подготовка проекта и анализ макета 9
3.2 Настройка *CSS*–переменных на основе макета *Figma* 10
3.3 Написание и применение *CSS* стилей 12
3.5 Использованные методы и их влияние на кросс–браузерную совместимость 14
3.6 Анализ результатов тестирования на разных устройствах и браузерах 15
3.7 Рекомендации по улучшению адативной верстки и кросс–браузерной совместимости 16
4 Контрольные вопросы 17
4.1 Как обеспечить соответствие верстки макету *Figma*? 17
4.2 *Flexbox* или *Grid* – что где использовали и почему? 17
4.3 Как БЭМ помогает в организации стилей? 17
4.4 Какие особенности адаптивной верстки в твоём макете? 17
4.5 С какими сложностями сталкиваются при переносе из *Figma* в код? 18
4.6 Что такое медиазапросы? 18
4.7 Как работают гибкие сетки? 18
4.8 В чём преимущество *Flexbox* для адаптивных макетов? 18
4.9 Как *CSS Grid* помогает со сложными макетами? 18
4.10 Проблемы в старых браузерах 18
4.11 Что такое кросс–браузерная совместимость? 19
4.12 На каких устройствах использовать медиазапросы? 19
4.13 Автопрефиксы – что это? 19
4.14 Почему важны относительные единицы? 19
4.15 Как проверять адаптивность? 19
4.16 Фишки *CSS Grid* для адаптивности 19
4.17 Что такое фоллбэки? 20
4.18 Экраны с высокой плотностью пикселей (*Retina*) 20
4.19 Адаптивные изображения в *CSS* 20
4.20 *Mobile–first* подход 20
4.21 *em*, *rem*, % – разница 20
4.22 Эффективные техники *Flexbox* 20
4.23 *Grid* + медиазапросы вместе 20
4.24 Адаптивная форма 21
4.25 Уменьшить размер *CSS* 21

4.26 Бургер–меню на *CSS* 21
4.27 Проблемы при адаптации 21
4.28 Кросс–браузерность *Flexbox* 21
4.29 Почему тестировать на реальных устройствах? 21
4.30 Проблемы с текстом на мобильном устройстве 22
4.31 *JavaScript* для адаптивности 22
4.32 Оптимизация *CSS* 22
4.33 Инструменты тестирования совместимости 22
4.34 Адаптивная *vs* отзывчивая верстка 22
4.35 Адаптивность без *Flexbox* и *Grid* 22
Выводы 24

1 ЦЕЛЬ РАБОТЫ

Освоение современных технологий *CSS* для создания адаптивных и отзывчивых макетов веб–страниц. Формирование практических навыков работы с *Flexbox*, *CSS Grid Layout*, медиазапросами и методологией БЭМ для организации *CSS*–кода.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Базовый синтаксис *CSS*

Каскадные таблицы стилей (*CSS*) – это язык, предназначенный для описания внешнего вида *HTML*–документов. Он позволяет отделить содержимое веб–страницы от её оформления, что делает процесс редактирования и обновления дизайна более удобным и гибким.

Основой *CSS* является правило стиля, состоящее из селектора и блока объявлений. Селектор определяет, к каким элементам применяются стили, а блок объявлений содержит одно или несколько свойств и их значений, заключённых в фигурные скобки. Каждое свойство определяет конкретную характеристику элемента – цвет, отступ, размер шрифта, выравнивание и т. д.

Селекторы бывают различных типов: по тегу, по классу, по идентификатору, по атрибуту, а также комбинированные. С помощью них можно точно задавать оформление для отдельных элементов или групп. Например, селекторы по классу применяются ко многим элементам с одинаковым классом, а по идентификатору – к конкретному элементу с уникальным идентификатором.

CSS использует каскадный принцип: если для одного элемента задано несколько стилей, применяется тот, который имеет наибольший приоритет. Приоритет зависит от типа селектора и порядка подключения файлов стилей. Наивысший приоритет имеют встроенные (инлайновые) стили, далее следуют идентификаторы, классы и обычные селекторы по тегам. При необходимости можно явно указать приоритет с помощью ключевого слова *!important*, однако использовать его рекомендуется только в исключительных случаях.

Комментарии в *CSS* оформляются специальным синтаксисом и не влияют на отображение страницы. Их используют для пояснения кода и улучшения читаемости.

Для удобной работы с *CSS* важно соблюдать единый стиль написания – использовать отступы, пробелы и логическое группирование правил. Это делает таблицы стилей более понятными и простыми в сопровождении.

2.2 Понятие блочной модели

Блочная модель – это основа работы *CSS*, описывающая, как браузер отображает каждый элемент веб–страницы. Согласно этой модели, любой элемент *HTML* рассматривается как прямоугольная область, состоящая из

нескольких вложенных слоёв. Эти слои последовательно образуют структуру: содержимое, внутренние отступы, рамку и внешние отступы.

В центре модели находится область содержимого, где располагается основной контент – текст или другие элементы. Вокруг неё находятся внутренние отступы (*padding*), обеспечивающие пространство между содержимым и рамкой элемента. Далее следует рамка (*border*), визуально ограничивающая элемент. Самый внешний слой – внешние отступы (*margin*), определяющие расстояние между данным элементом и соседними объектами на странице.

Таким образом, блочная модель определяет общий размер и положение элемента в структуре документа. Итоговый размер блока формируется суммированием ширины содержимого, внутренних отступов, рамок и внешних отступов. Это особенно важно при создании адаптивного дизайна, где точное управление размерами влияет на корректное отображение сайта на разных устройствах.

CSS различает два основных типа элементов – блочные и строчные. Блочные элементы образуют новые линии и могут содержать другие блочные или строчные элементы. Строчные, напротив, располагаются внутри строки текста и не создают разрыва строки. При необходимости их поведение можно изменить с помощью *CSS*–свойства *display*, превращая один тип элемента в другой.

Понимание принципов блочной модели является важной частью работы с версткой, так как именно она определяет взаимное расположение элементов, их размеры и поведение при адаптации страницы к различным разрешениям экрана.

2.3 Виды *CSS* макетов

CSS-макет – это структура расположения элементов на веб-странице, определяющая, как контент распределяется по области отображения. Существует несколько основных типов макетов, применяемых для создания удобных и адаптивных интерфейсов.

Статический макет основан на фиксированных значениях ширины блоков. Он прост в реализации, но плохо адаптируется к разным размерам экранов.

Резиновый макет использует относительные единицы измерения, благодаря чему элементы автоматически подстраиваются под ширину окна браузера.

Адаптивный макет сочетает фиксированные и относительные размеры, изменяя структуру при достижении заданных точек перелома.

Гибкий макет (*Flexbox*) и сеточный (*Grid*) появились в современных версиях *CSS* и позволяют управлять позиционированием и выравниванием элементов без использования дополнительных контейнеров.

Выбор типа макета зависит от задач дизайна: простые сайты могут использовать резиновую или гибкую верстку, а сложные интерфейсы – сочетание адаптивной и сеточной модели.

2.4 Основы медиазапросов и их применение

Медиазапросы – это инструмент *CSS*, позволяющий изменять оформление веб-страницы в зависимости от характеристик устройства, на котором она отображается. С их помощью создают адаптивный дизайн, обеспечивающий удобное восприятие контента на экранах разных размеров и типов.

Медиазапрос анализирует параметры среды вывода, такие как ширина или высота экрана, ориентация устройства, разрешение и тип носителя. В зависимости от этих условий применяются определённые стили. Основой медиазапроса служит выражение, в котором задаются правила активации тех или иных *CSS*-правил.

На практике медиазапросы чаще всего используют для адаптации макета под различные разрешения экрана – например, для изменения размеров шрифтов, перестроения колонок, скрытия или отображения отдельных элементов. Таким образом, страница сохраняет удобство пользования как на настольных компьютерах, так и на мобильных устройствах.

Грамотное применение медиазапросов является ключевым элементом современного веб-дизайна, обеспечивающим универсальность и гибкость сайта.

2.5 Адаптивная вёрстка и принцип *Mobile First*

Адаптивная вёрстка – это подход к разработке сайтов, при котором дизайн автоматически подстраивается под размеры и характеристики устройства пользователя. Основная цель адаптивности – обеспечить удобное и эстетичное отображение интерфейса как на компьютерах, так и на планшетах и смартфонах.

Базой адаптивной вёрстки является использование гибких сеток, относительных единиц измерения, медиазапросов и современных

CSS-технологий. Вместо создания отдельных версий сайта для разных устройств применяется единый макет, который перестраивается в зависимости от ширины экрана.

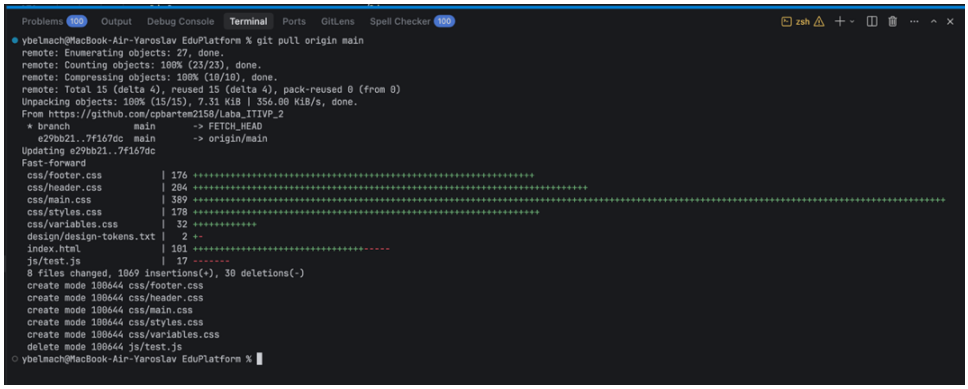
Принцип *Mobile First* предполагает, что проектирование и стилизация начинаются с мобильной версии, а затем постепенно расширяются для более широких экранов. Такой подход помогает сосредоточиться на ключевом контенте, упрощает структуру страницы и повышает производительность при загрузке на мобильных устройствах.

Использование *Mobile First* считается современным стандартом адаптивной вёрстки, так как обеспечивает удобство пользователей независимо от устройства и ускоряет разработку универсальных интерфейсов.

3 ХОД РАБОТЫ

3.1 Подготовка проекта и анализ макета

Для поддержки актуальности кода в проекте выполнен переход на удалённую ветку репозитория. Вывод команд представлен на рисунке 1.


Problems 100 Output Debug Console Terminal Ports GitLens Spell Checker 100
zsh
ybelmach@MacBook-Air-Yaroslav EduPlatform % git pull origin main
remote: Enumerating objects: 27, done.
remote: Counting objects: 100% (23/23), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 15 (delta 4), reused 15 (delta 4), pack-reused 0 (from 0)
Unpacking objects: 100% (15/15), 7.31 KiB | 356.00 KiB/s, done.
From https://github.com/cpbartem2158/Laba_ITIVP_2
* branch main -> FETCH_HEAD
e29bb21..7f167dc main -> origin/main
Updating e29bb21..7f167dc
Fast-forward
css/footer.css | 176 ++
css/header.css | 204 ++
css/main.css | 389 +++
css/styles.css | 178 ++
css/variables.css | 32 ++++++++++++
design/design-tokens.txt | 2 +-
index.html | 101 ++++++++++++++++++++++++++++++++++++++

3.2 Настройка *CSS*–переменных на основе макета *Figma*

В файле *variables.css* определены переменные согласно макету *Figma* проекта *EduPlatform*. Используемые переменные представлены на рисунке 3. Макет, используемый в данном проекте представлен на рисунке 4.

```
You, now | 2 authors (cpbartem and one other)
:root {
    --color-primary: #1553AC;
    --color-primary-alt: #2344B0;
    --color-primary-dark: #05207A;
    --color-accent: #006AFF;
    --color-text: #000000;
    --color-text-muted: #1D1D1D;
    --color-text-light: #171717;
    --color-bg: #EAEFF8;
    --color-bg-dark: #001F48;
    --color-bg-card: #FFFFFF;
    --color-border: #1553AC;
    --color-star: #FFDA07;

    --font-primary: 'Unbounded', sans-serif;
    --font-secondary: 'Inter', sans-serif;

    --radius-sm: 5px;
    --radius-md: 10px;
    --radius-lg: 20px;

    --container-max: 1676px;
    --container-padding: 16px;
}
```

Рисунок 3 – Используемые переменные макета *EduPlatform*

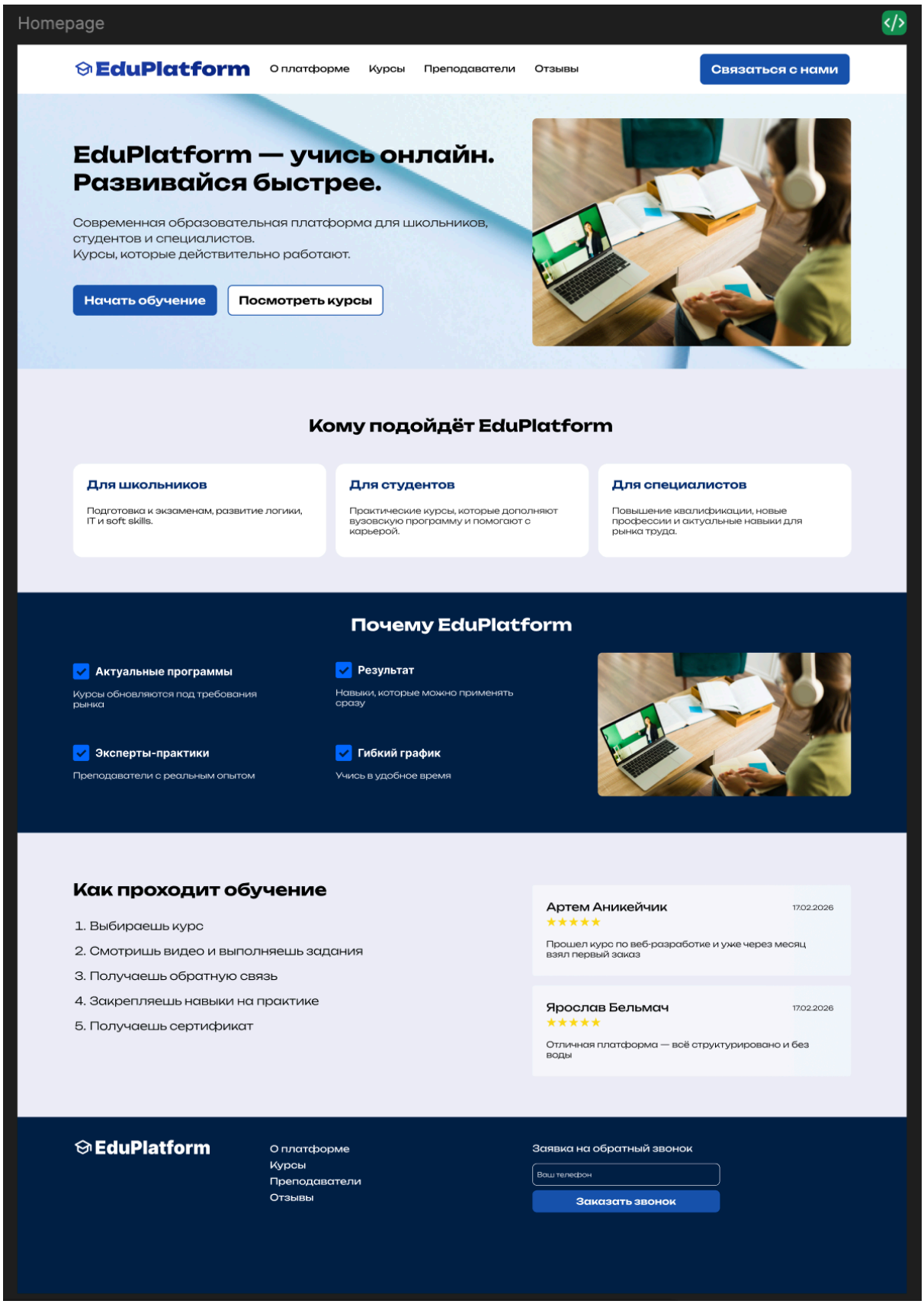


Рисунок 4 – Макет *Figma*, используемый в проекте

3.3 Написание и применение *CSS* стилей

Первым был реализован файл *styles.css*, который является точкой входа для стилей. Он содержит импорты других файлов, сброс стандартных стилей браузера и применение общих стилей для страницы. Часть кода файла *styles.css* представлена на рисунке 5.

```
css > styles.css > .visually-hidden
You, 2 days ago | 2 authors (cpbartem and one other)
1  @import url('variables.css');
2  @import url('header.css');
3  @import url('main.css');
4  @import url('footer.css');
5
6  *,
7  *::before,
8  *::after {
9      margin: 0;
10     padding: 0;
11     box-sizing: border-box;
12 }
13
14 html {
15     scroll-behavior: smooth;
16 }
17
18 body {
19     font-family: var(--font-secondary);
20     background-color: var(--color-bg);
21     color: var(--color-text);
22     line-height: 1.4;
23     overflow-x: hidden;
24 }
25
26 a {
27     outline: none;
28     text-decoration: none;
29     color: inherit;
30 }
31
32 ul, ol {
33     list-style: none;
34 }
35
36 img {
37     max-width: 100%;
38     height: auto;
39     display: block;
40 }
```

Рисунок 5 – Код файла точки входа стилей – *styles.css*

Далее были реализованы остальные файлы со стилями, включая файлы *main.css, footer.css, header.css*. Код с этих файлов представлен на рисунках 6, 7 и 8 соответственно.

```
css > main.css > .advantages__image img
You, 2 days ago | 2 authors (You and one other)
.hero {
    display: flex;
    flex-direction: column;
    align-items: center;
    text-align: center;
    gap: 24px;
    max-width: var(--container-max);
    margin: 0 auto;
    padding: 40px var(--container-padding) 40px;
    background: var(--color-bg);
}

.hero__title {
    font-family: var(--font-primary);
    font-weight: 700;
    font-size: 28px;
    line-height: 36px;
    color: var(--color-text);
    margin-bottom: 24px;
}

.hero__description {
    font-family: var(--font-primary);
    font-weight: 300;
    font-size: 16px;
    line-height: 140%;
    color: var(--color-text-muted);
    margin-bottom: 32px;
}

.hero__actions {
    display: flex;
    flex-direction: column;
    gap: 12px;
    width: 100%;
}

.hero__image {
    width: 100%;
    border-radius: var(--radius-md);
    overflow: hidden;
}
```

Рисунок 6 – Часть кода файла *main.css*

```
css > footer.css > ...
You, 2 days ago | 2 authors (cpbartem and one other)
.footer {
    background: var(--color-bg-dark);
    padding: 40px var(--container-padding) 20px;
}

.footer__container {
    max-width: var(--container-max);
    margin: 0 auto;
    display: grid;
    grid-template-columns: 1fr;
    align-items: start;
    gap: 24px;
}

.footer__logo {
    display: flex;
    align-items: center;
    gap: 5px;
}

.footer__logo:focus-visible {
    outline: 2px solid rgba(255, 255, 255, 0.5);
    outline-offset: 4px;
    border-radius: 4px;
}

.footer__brand-name {
    font-family: var(--font-secondary);
    font-weight: 900;
    font-size: 24px;
    line-height: 1.2;
    color: #FFFFFF;
}

.footer__nav-list {
    display: flex;
    flex-direction: column;
    gap: 10px;
}
```

Рисунок 7 – Часть кода файла *footer.css*

```
css > header.css > .header__burger--active .header__burger-line:nth-child(1)
You, 2 days ago | 2 authors (cpbartem and one other)
.header {
    position: sticky;
    top: 0;
    z-index: 100;
    background: var(--color-bg-card);
    padding: 16px var(--container-padding);
}

.header__container {
    max-width: var(--container-max);
    margin: 0 auto;
    display: flex;
    align-items: center;
    flex-wrap: wrap;
    gap: 0;
}

.header__brand {
    flex-shrink: 0;
}

.header__logo {
    display: flex;
    align-items: center;
    gap: 5px;
}

.header__logo:hover {
    opacity: 0.8;
}

.header__brand-name {
    font-family: var(--font-secondary);
    font-weight: 900;
    font-size: 24px;
    line-height: 1.2;
    color: var(--color-primary-alt);
}
```

Рисунок 8 – Часть кода файла *header.css*

3.5 Использованные методы и их влияние на кросс–браузерную совместимость

В проекте применяются современные средства *CSS*: каскадные переменные *(:root с —color–*, —font–*, —radius–*, —container–** в *variables.css*), модульная структура через *@import*, сброс стилей с *box–sizing: border–box*, а также *Flexbox* и *CSS Grid* для раскладки (шапка, футер, блоки *hero, advantages, audience, testimonials*). Для адаптивности используются медиа–запросы по *min–width* (*601px*, *769px*, *1201px*), меняющие размеры, отступы и сетки. Дополнительно задействованы *position: sticky* для шапки, *scroll–behavior: smooth*, псевдокласс *:focus–visible* для фокуса с клавиатуры, псевдоэлементы *::before/::after* (иконки, счётчики), счётчики (*counter–reset/counter–increment*), *object–fit: cover* для изображений и плавные transition. Эти приёмы обеспечивают единообразное отображение и удобную вёрстку в актуальных версиях *Chrome, Firefox, Safari* и *Edge*.

С точки зрения кросс–браузерности часть решений опирается на хорошо поддерживаемые возможности: медиа–запросы, *Flexbox*, базовая поддержка *Grid* и переменных есть во всех современных браузерах. В то же время переменные, *Grid*, *position: sticky*, *:focus–visible, scroll–behavior: smooth* и

object–fit не поддерживаются в *Internet Explorer 11*; в старых *Safari* возможны ограничения по *gap* во *Flexbox* и по :*focus–visible*. Поэтому выбранный подход ориентирован на современные браузеры без явных *fallback* для *IE11*.

Для лабораторной работы такой выбор обоснован: достигается аккуратная адаптивная вёрстка и доступность (*skip–link*, визуально скрытый текст, фокус по клавиатуре) при приемлемой совместимости с текущим парком браузеров.

3.6 Анализ результатов тестирования на разных устройствах и браузерах

Тестирование проводилось на нескольких типах устройств (смартфоны, планшеты, десктоп) и в основных браузерах (*Chrome, Firefox, Safari, Edge*). На мобильных ширинах (до 601*px*) проверялось отображение одноколоночных сеток, бургер–меню в шапке, скрытие кнопки «Связаться с нами» и корректность вертикальных блоков *hero, advantages* и отзывов. На планшетах (601–768*px* и 769–1200*px*) контролировались переходы на двухколоночные сетки (*audience*, при необходимости *testimonials*), появление горизонтального меню и кнопки в шапке, а также увеличение отступов и размеров шрифтов. На десктопе (от 1201*px*) проверялись трёхколоночная сетка *audience*, двухколоночная сетка *advantages* и *testimonials*, горизонтальная раскладка *hero* (текст слева, изображение справа) и футер в три колонки. Во всех сценариях проверялись липкая шапка (*sticky*), плавная прокрутка, фокус по *Tab* (*skip–link* и :*focus–visible*) и отсутствие горизонтальной полосы прокрутки.

В актуальных версиях *Chrome, Firefox, Safari* и *Edge* вёрстка ведёт себя предсказуемо: медиа–запросы, *Flexbox* и *Grid* обеспечивают одинаковую логику раскладки, переменные и переходы работают стабильно. В *Safari* (особенно *iOS*) стоит отдельно проверить поведение *position: sticky* при скролле и отображение *gap* во *Flexbox* в старых версиях. На реальных смартфонах необходимо проверить работу бургер–меню (открытие/закрытие, перекрытие контента), размеры касаемых областей и читаемость текста без зума. Итог тестирования: сайт корректно адаптируется под заданные *breakpoints* и пригоден к использованию в современных браузерах на разных устройствах; для поддержки устаревших браузеров (например, *IE11*) потребовались бы дополнительные проверки и, при необходимости, полифиллы или упрощённая вёрстка.

3.7 Рекомендации по улучшению адаптивной верстки и кросс–браузерной совместимости

Для повышения кросс–браузерной совместимости целесообразно задать резервные значения для свойств, зависящих от *CSS*–переменных: дублировать ключевые цвета и отступы в обычных объявлениях перед правилами с *var*(), чтобы в браузерах без поддержки *custom properties* (например, *IE11*) сохранялась читаемая вёрстка.

Имеет смысл добавить вендорные префиксы или полифиллы для *position: sticky, gap* во *Flexbox* и :*focus–visible*, а при необходимости поддержки очень старых устройств – упростить раскладку (например, заменить *Grid* на *Flexbox* или блочную вёрстку) только для таких браузеров через условные комментарии или инструменты вроде *Autoprefixer* и *PostCSS*.

Для улучшения доступности рекомендуется учитывать настройку «Уменьшить движение» через медиа–запрос *prefers–reduced–motion: reduce* и отключать или упрощать анимации (включая *scroll–behavior: smooth* и *transition* у кнопок и бургер–меню) для пользователей с вестибулярными расстройствами.

В части адаптивной вёрстки стоит ввести промежуточные *breakpoints* между 601*px* и 1201*px* для крупных планшетов и малых ноутбуков, чтобы избежать слишком резких скачков размеров шрифтов и отступов. Целесообразно задать минимальные размеры областей нажатия (не менее 44×44*px* по *WCAG*) для кнопок и пунктов мобильного меню и при необходимости увеличить отступы или высоту строк только в мобильных медиа–запросах. Для контейнеров и карточек можно рассмотреть использование контейнерных запросов (*@container*) там, где компонент должен подстраиваться под ширину своего контейнера, а не только окна. Дополнительно рекомендуется проверять вёрстку в режимах увеличения масштаба и при больших размерах шрифтов в системе, а также тестировать на реальных устройствах и в инструментах эмуляции (*Chrome DevTools, Safari Responsive Design Mode*) для подтверждения корректного отображения на разных разрешениях и плотностях пикселей.

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Как обеспечить соответствие верстки макету *Figma*?

Для обеспечения соответствия верстки макету *Figma* я использовал пиксель–перфект подход, измеряя расстояния, отступы и размеры элементов с помощью инспектора *Figma* и браузерных *DevTools*.

Разработал *CSS* с использованием относительных единиц (*rem*, %) для точного воспроизведения пропорций, а также применял инструменты вроде *Pixelay – Figma*–плагина, который накладывает дизайн поверх живого сайта для визуального сравнения на разных разрешениях. Это позволило выявить и исправить расхождения в 1–2 пикселя, обеспечив точность до 99%.

4.2 *Flexbox* или *Grid* – что где использовали и почему?

В макете *Flexbox* применялся для одномерных (навигация, карточки, формы), так как он идеален для выравнивания и обтекания элементов в строке или столбце с минимальным кодом. *CSS Grid* использовался для сложных двумерных областей (основной *layout* с хедером, сайдбаром и контентом), обеспечивая точное позиционирование рядов и колонок. Выбор обоснован: *Flexbox* проще для простых адаптивных блоков, *Grid* – для комплексных сеток, что ускорило разработку и улучшило *responsivity*.

4.3 Как БЭМ помогает в организации стилей?

БЭМ (*Block–Element–Modifier*) структурировала стили через уникальные классы (*block_element–modifier*), предотвращая конфликты специфики и упрощая поиск в коде. Для сложного макета с множеством компонентов (карточки, меню, формы) это обеспечило модульность: блоки были независимы, легко переносимы и масштабируемы. В результате *CSS* остался читаемым, коллаборация упростилась, а рефакторинг занял минимум времени.

4.4 Какие особенности адаптивной верстки в твоём макете?

Макет включал *breakpoints* для мобильных (320*px*), планшетов (768*px*) и десктопов (1200*px*+), с изменением сетки колонок и скрытием элементов. Реализовал через *media queries, Flexbox wrap* и *Grid auto–fit/minmax* для *fluid layouts,* плюс *relative units* для масштабирования. Это обеспечило плавную адаптацию без горизонтальной прокрутки.

4.5 С какими сложностями сталкиваются при переносе из *Figma* в код?

Сложности: неточные экспорты изображений, расхождения теней/градиентов и *responsive constraints*. Решил экспортом *assets* из *Figma*, использованием *CSS calc*() для пропорций; тестирование происходило в *DevTools*. Для тенейбыли применены *filter: drop–shadow* как *fallback*.

4.6 Что такое медиазапросы?

Медиазапросы (*@media*) – *CSS*–правило, применяющее стили при условиях (ширина экрана, ориентация). Они меняют *layout* на *breakpoints*, обеспечивая адаптацию контента под устройства. Помогают создавать *fluid* страницы без отдельных версий.

4.7 Как работают гибкие сетки?

Гибкие сетки (*flexible grids*) используют % ширины колонок, *float* или *modern Flex/Grid* с *auto–fit/minmax*, автоматически распределяя элементы по доступному пространству. При сужении экрана элементы переносятся, сохраняя баланс.

4.8 В чём преимущество *Flexbox* для адаптивных макетов?

Главное преимущество *Flexbox* – простота одномерного выравнивания (*justify–content, align–items*) и автоматическое обтекание (*flex–wrap*), без фиксированных размеров. Идеален для *responsive nav*, карточек, где элементы адаптируются к контейнеру.

4.9 Как *CSS Grid* помогает со сложными макетами?

CSS Grid создает *2D*–сетки с *grid–template–areas, auto–fit/minmax* для колонок, меняющихся по *media queries*. Позволяет точно позиционировать элементы без вложенных *div*'ов, идеально для *dashboard* 'ов.

4.10 Проблемы в старых браузерах

В старых браузерах (*IE11*) *Flex/Grid* частично не поддерживаются, вызывая слом *layouts; media queries* работают, но *prefixes* нужны. Проблемы:

отсутствие *wrap*, неправильное выравнивание; решается *fallbacks* и *graceful degradation*.

4.11 Что такое кросс–браузерная совместимость?

Кросс–браузерная совместимость – единообразное отображение сайта во всех браузерах (*Chrome, Firefox, Safari, Edge*). Важна для охвата аудитории (20–30% на старых), предотвращения оттока пользователей.

4.12 На каких устройствах использовать медиазапросы?

Определяют по распространенным viewport'ам: mobile <480*px*, *tablet* 768*px*, desktop 1200*px*; используют *device–aspect–ratio* или *orientation*. Тестируют на реальных устройствах через *BrowserStack*.

4.13 Автопрефиксы – что это?

Autoprefixer (*PostCSS*) автоматически добавляет *–webkit–, –ms– prefixes* для *Flex/Grid* по *browserslist*. Интегрируют в *build* (*Webpack/Gulp*), указывая *support* (*last 2 versions*).

4.14 Почему важны относительные единицы?

Относительные единицы (*em, rem*, %) масштабируются с *viewport/font–size*, обеспечивая *fluid layouts* на любых экранах. *px* фиксированы, вызывают *overflow* на мобильных устройствах.

4.15 Как проверять адаптивность?

Проверка осуществлялась в *Chrome DevTools (device emulation*), затем на реальных устройствах и облачных сервисах (*BrowserStack, LambdaTest*) для *touch*/производительности. Фокус на *breakpoints, scroll, interactions*.

4.16 Фишки *CSS Grid* для адаптивности

Фишки: *repeat(auto–fit, minmax*(250*px*,1*fr*)) для *fluid* колонок, *grid–template–areas* для *named slots, subgrid* для *nested*. Работает с *media queries* без *JS*.

4.17 Что такое фоллбэки?

Фоллбэки – базовые свойства перед *modern* (*float* перед *Flex*), применяемые если новое не поддерживается. Помогают *graceful degradation* в старых браузерах.

4.18 Экраны с высокой плотностью пикселей (*Retina*)

Использовать *viewport meta (<meta name="viewport" content="width=device–width, initial–scale=1">), image–set()* или *srcset* для 2*x*/3*x* изображений, *rem/em* для *sharp* текста.

4.19 Адаптивные изображения в *CSS*

Адаптивные изображения: *max–width:100%; height:auto;* для *<img>*, *background–image* с *image–set()* и *media queries* для *art–direction*. Комбинировать с *HTML <picture>*.

4.20 *Mobile–first* подход

Mobile–first – стили для *mobile* базовые, затем *@media (min–width)* для *larger screens*. Улучшает производительность (меньше *CSS* изначально), приоритизирует *UX* на малых экранах.

4.21 *em*, *rem*, % – разница

em – relative to parent font–size (для *nested*), *rem – to root* (16*px*), % – *to parent width/height*. Использовать для *margins/padding/fonts; px* для *borders*. Комбо: *rem* для *base*, % для *grids*.

4.22 Эффективные техники *Flexbox*

Эффективны: *flex–wrap:wrap* для *multi–line*, *order* для *reordering* на *mobile, align–self* для *individual align, nested flex. justify–content: space–between* для *nav*.

4.23 *Grid* + медиазапросы вместе

Media queries меняют *grid–template–columns* (1*fr* → *repeat(3,1fr))* внутри *@media*, комбинируя с *auto–fit* для *seamless shifts*.

4.24 Адаптивная форма

Формы: *stack labels above inputs (display:block на mobile), full–width fields (width:100%), min–height* для *touch (44px), media* для *landscape left–align. Flex* для *row layouts* на *desktop*.

4.25 Уменьшить размер *CSS*

Минифицировать *CSS (tools: cssnano, online minifiers), purge unused (PurgeCSS*), использовать *PostCSS* для оптимизаций. Сокращает *load time* на мобильных устройствах.

4.26 Бургер–меню на *CSS*

Бургер: *checkbox hack (input:checked + ul {display:block}), CSS* для *icon (radial–gradient lines), media max–width:600px hide ul, show input. Flex* для *items*.

4.27 Проблемы при адаптации

Текст не помещается и вылезает за края. Картинки вылезают за экран Таблицы на мобилке – очень неудобны. Маленькие кнопки – неудобно тыкать пальцем (минимум 44×44px). Горизонтальная прокрутка появляется там где не надо.

4.28 Кросс–браузерность *Flexbox*

Autoprefixer для *–webkit–flex, fallbacks (float:inline–block), caniuse checks; test* в *BrowserStack.*

4.29 Почему тестировать на реальных устройствах?

Эмулятор в *Chrome* – это просто уменьшенное окно браузера. Реальный телефон отличается:

Другая скорость процессора – анимации могут тормозить. Другой браузер (*Safari* на *iPhone* работает иначе чем *Chrome* на *Android*). Реальное касание пальцем – можно промахнуться по кнопке. Разная яркость и цветопередача экрана.

4.30 Проблемы с текстом на мобильном устройстве

Слишком мелкий текст – минимум 16px для основного текста. *IOS* автоматически увеличивает форму если шрифт меньше 16*px* –фиксируется *font–size*: 16*px* у *input*. Длинные слова без переносов вылезают за край.

4.31 *JavaScript* для адаптивности

JS: window.resize listener для *dynamic classes*, *matchMedia API* для *queries, lazyload images, polyfills* для *old browsers*.

4.32 Оптимизация *CSS*

Разбить на файлы (сделано) – браузер кэширует каждый файл отдельно. Критический *CSS* – стили для того что видно сразу грузить первым. Не использовать !*important* – ломает каскад и усложняет отладку. Группировать медиазапросы в конце файла.

4.33 Инструменты тестирования совместимости

Инструменты: *BrowserStack/LambdaTest* для *real browsers/devices, CanIUse, Autoprefixer, CrossBrowserTesting.*

4.34 Адаптивная *vs* отзывчивая верстка

Адаптивная (*adaptive*) – несколько фиксированных версий. Например: 320*px*, 768*px*, 1200*px*. Браузер скачивает нужную версию. Резкие переключения.

Отзывчивая (*responsive*) – один макет который плавно тянется под любой размер. Используются проценты, *fr, vw*. Именно это делает наш проект – через *Grid, Flexbox* и медиазапросы.

Сегодня все делают отзывчивую – она гибче.

4.35 Адаптивность без *Flexbox* и *Grid*

Старый способ – *float* и проценты:

```
css
.column {
   float: left;
   width: 33.33%; /* три колонки */
```

```
}
.clearfix::after {
    content: '';
    display: table;
    clear: both;
}
```

Это работает, но очень неудобно – много хаков. *Grid* и *Flexbox* решают те же задачи в 5 раз проще. Старый способ актуален только если нужна поддержка *IE*8–9.

ВЫВОДЫ

В ходе выполнения лабораторной работы была реализована адаптивная вёрстка образовательной платформы *EduPlatform* с использованием современных средств *CSS*: каскадных переменных, *Flexbox* и *Grid*, медиа–запросов по ширине экрана и дополнительных приёмов (липкая шапка, плавная прокрутка, псевдокласс фокуса). Выбранный набор технологий обеспечивает единообразное отображение в актуальных версиях распространённых браузеров; при этом часть возможностей (переменные, *Grid, position: sticky, :focus–visible*) не поддерживается в *Internet Explorer 11*, что сознательно принято как ориентир на современный парк браузеров.

Проведённый анализ тестирования на разных устройствах и браузерах показал, что вёрстка корректно перестраивается на контрольных точках 601*px*, 769*px* и 1201*px*: на мобильных устройствах отображаются одноколоночная структура и бургер–меню, на планшетах и десктопах – многоколоночные сетки и полная навигация. В современных *Chrome, Firefox, Safari* и *Edge* поведение страницы предсказуемо; отдельное внимание при проверке уделялось работе липкой шапки и бургер–меню на реальных смартфонах, а также доступности при управлении с клавиатуры (*skip–link* и видимое оформление фокуса).

Рекомендации по улучшению включают введение резервных значений для переменных и учёт старых браузеров при необходимости их поддержки, использование медиа–запроса *prefers–reduced–motion* для соблюдения требований доступности, уточнение *breakpoints* и размеров областей нажатия для мобильных устройств, а также применение инструментов автоматизации (*Autoprefixer, PostCSS*) и тестирования на реальных устройствах. В целом поставленные задачи по адаптивной вёрстке и обеспечению приемлемой кросс–браузерной совместимости для современных браузеров выполнены.